1) Recursive Approach:

```
#include <iostream>
#include <algorithm>
using namespace std;
struct Node {
   int data;
   Node* left;
   Node* right;
   Node(int val) {
      data = val;
      left = NULL;
      right = NULL;
   }
};
Node* insert(Node* root, int key) {
   if (root == NULL) {
      return new Node(key);
   }
   if (key < root->data) {
      root->left = insert(root->left, key);
   } else if (key > root->data) {
      root->right = insert(root->right, key);
   }
   return root;
}

void inorder(Node* root) {
   if (root != NULL) {
      inorder(root->left);
      cout << root->data << " ";
      inorder(root->right);
   }
}

void preorder(Node* root) {
   if (root != NULL) {
      cout << root->data << " ";
      preorder(root->left);
      preorder(root->right);
   }
}

void postorder(Node* root) {
```

```
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}

int height(Node* root) {
    if (root == NULL) {
        return 0;
    }
    int leftHeight = height(root->left);
    int rightHeight = height(root->right);
    return max(leftHeight, rightHeight) + 1;
}


int countLeaves(Node* root) {
    if (root == NULL) {
        return 0;
    }
    if (root->left == NULL && root->right == NULL) {
        return 1;
    }
    return countLeaves(root->left) + countLeaves(root->right);
}

int countInternal(Node* root) {
    if (root == NULL || (root->left == NULL && root->right == NULL)) {
        return 0;
    }
    return 1 + countInternal(root->left) + countInternal(root->right);
}

int main() {
    Node* root = NULL;
    int n,val;
    cout<<"Enter the Number of Nodes: "<<endl;
    cin>>n;
    for (int i = 0; i < n; i++) {
            cout<<"Enter the Value of Nodes: "<<endl;
            cin>>val;
               root = insert(root, val);
```

```
    }

    cout << "Traversals" << endl;
    cout << "In-order:  ";
    inorder(root);
    cout << endl;
    cout << "Pre-order: ";
    preorder(root);
    cout << endl;
    cout << "Post-order:";
    postorder(root);
    cout << endl << endl;
    cout << "Requested Operations" << endl;
    cout << "Height of the tree: " << height(root) << endl;
    cout << "Number of leaf nodes: " << countLeaves(root) << endl;
    cout << "Number of internal nodes: " << countInternal(root) << endl;
    return 0;
}
```

Output:

```
Enter the Number of Nodes: 9
Enter the Value of Nodes: 30
Enter the Value of Nodes: 25
Enter the Value of Nodes: 36
Enter the Value of Nodes: 24
Enter the Value of Nodes: 28
Enter the Value of Nodes: 29
Enter the Value of Nodes: 31
Enter the Value of Nodes: 42
Enter the Value of Nodes: 49
Traversals
In-order:  24 25 28 29 30 31 36 42 49
Pre-order: 30 25 24 28 29 36 31 42 49
Post-order:24 29 28 25 31 49 42 36 30
Height of the tree: 4
Number of leaf nodes: 4
Number of internal nodes: 5
```

2) Iterative Approach:

```
#include <iostream>
#include <algorithm>
#include <stack>
#include <queue>
using namespace std;
```

```
struct Node {
   int data;
   Node* left;
   Node* right;
   Node(int val) {
      data = val;
      left = NULL;
      right = NULL;
   }
};
Node* insert(Node* root, int key) {
   Node* newNode = new Node(key);
   if (root == NULL) return newNode;
   Node* curr = root;
   Node* parent = NULL;
   while (curr != NULL) {
      parent = curr;
      if (key < curr->data)
         curr = curr->left;
      else if (key > curr->data)
         curr = curr->right;
      else {
         delete newNode;
         return root;
      }
   }
   if (key < parent->data)
      parent->left = newNode;
   else
      parent->right = newNode;
   return root;
}
void inorder(Node* root) {
   stack<Node*> s;
   Node* curr = root;
   while (curr != NULL || !s.empty()) {
      while (curr != NULL) {
         s.push(curr);
         curr = curr->left;
      }
      curr = s.top();
      s.pop();
      cout << curr->data << " ";
```

```
        curr = curr->right;
    }
}
void preorder(Node* root) {
    if (root == NULL) return;
    stack<Node*> s;
    s.push(root);
    while (!s.empty()) {
        Node* curr = s.top();
        s.pop();
        cout << curr->data << " ";
        if (curr->right) s.push(curr->right);
        if (curr->left) s.push(curr->left);
    }
}
void postorder(Node* root) {
    if (root == NULL) return;
    stack<Node*> s1, s2;
    s1.push(root);
    while (!s1.empty()) {
        Node* curr = s1.top();
        s1.pop();
        s2.push(curr);
        if (curr->left) s1.push(curr->left);
        if (curr->right) s1.push(curr->right);
    }
    while (!s2.empty()) {
        cout << s2.top()->data << " ";
        s2.pop();
    }
}

int height(Node* root) {
    if (root == NULL) return 0;
    queue<Node*> q;
    q.push(root);
    int h = 0;
    while (!q.empty()) {
        int size = q.size();
        h++;
        while (size--) {
            Node* curr = q.front();
            q.pop();
```

```
            if (curr->left) q.push(curr->left);
            if (curr->right) q.push(curr->right);
        }
    }
    return h;
}
int countLeaves(Node* root) {
    if (root == NULL) return 0;
    int count = 0;
    stack<Node*> s;
    s.push(root);
    while (!s.empty()) {
        Node* curr = s.top();
        s.pop();
        if (curr->left == NULL && curr->right == NULL) count++;
        if (curr->right) s.push(curr->right);
        if (curr->left) s.push(curr->left);
    }
    return count;
}
int countInternal(Node* root) {
    if (root == NULL) return 0;
    int count = 0;
    stack<Node*> s;
    s.push(root);
    while (!s.empty()) {
        Node* curr = s.top();
        s.pop();
        if (curr->left != NULL || curr->right != NULL) {
            count++;
            if (curr->right) s.push(curr->right);
            if (curr->left) s.push(curr->left);
        }
    }
    return count;
}
int main() {
    Node* root = NULL;
    int n, val;
    cout << "Enter the Number of Nodes: " << endl;
    cin >> n;
    for (int i = 0; i < n; i++) {
        cout << "Enter the Value of Node " << i + 1 << ": ";
```

```
        cin >> val;
        root = insert(root, val);
    }
    cout << "\nTraversals " << endl;
    cout << "In-order:   "; inorder(root); cout << endl;
    cout << "Pre-order:  "; preorder(root); cout << endl;
    cout << "Post-order: "; postorder(root); cout << endl;
    cout << "\nRequested Operations " << endl;
    cout << "Height of the tree: " << height(root) << endl;
    cout << "Number of leaf nodes: " << countLeaves(root) << endl;
    cout << "Number of internal nodes: " << countInternal(root) << endl;
    return 0;
}
```

Output:

```
Enter the Number of Nodes: 9
Enter the Value of Node 1: 21
Enter the Value of Node 2: 23
Enter the Value of Node 3: 24
Enter the Value of Node 4: 26
Enter the Value of Node 5: 27
Enter the Value of Node 6: 31
Enter the Value of Node 7: 32
Enter the Value of Node 8: 34
Enter the Value of Node 9: 29
Traversals
In-order:   21 23 24 26 27 29 31 32 34
Pre-order:  21 23 24 26 27 31 29 32 34
Post-order: 29 34 32 31 27 26 24 23 21
Requested Operations
Height of the tree: 8
Number of leaf nodes: 2
Number of internal nodes: 7
```